Guia de evoluções: ExpoCalc

Fatec ADS · Desenvolvimento de dispositivos móveis · 21/10/2026

Em duplas, vocês vão evoluir o ExpoCalc com três recursos novos, do mais simples ao mais desafiador. Façam no mínimo a Evolução 1.

Antes de começar

1. Clone o projeto: `git clone https://github.com/mrAndersonfeitosa/ExpoCalc-aula.git`
2. Entre na pasta e instale: `cd ExpoCalc-aula` e `npm install`
3. Rode: `npx expo start` e abra no Expo Go. Se a rede da sala falhar, use `npx expo start --tunnel`.
4. Faça duas contas e abra o histórico para ver o app funcionando.

Regras da dupla

- Uma pessoa digita e a outra revisa. Troquem de papel a cada evolução.
- Travou? Use as pistas em ordem (Pista 1, depois 2, depois 3).
- Depois de resolver, expliquem cada linha nova com as suas palavras.

Onde mexer

Arquivo	O que faz
`src/screens/HomeScreen.js`	Estado da calculadora e do histórico; botão "Ver histórico"
`src/screens/HistoricoScreen.js`	Mostra a lista com FlatList
`src/storage/historico.js`	Funções `salvar` e `carregar` do AsyncStorage

Evolução 1: Limpar histórico

A tela de histórico ganha um botão que apaga todas as contas, da tela e do celular. Conceitos: `useState`, `async/await` e `AsyncStorage.removeItem`.

Resultado esperado

- Ao tocar em "Limpar histórico", a lista mostra "Nenhuma conta ainda".
- Ao fechar e abrir o app, o histórico continua vazio.
- Depois de limpar, uma conta nova aparece sozinha no histórico (sem as antigas voltarem).

Pistas

1. A `HistoricoScreen` hoje só lê `route.params.historico`. Para a lista mudar na tela, ela precisa virar estado: `const [lista, setLista] = useState(route.params.historico)`.
2. Em `src/storage/historico.js`, crie `export async function limpar()` que chama `AsyncStorage.removeItem(CHAVE)`.
3. Adicione um `Button` na tela. No `onPress`, chame `await limpar()` e depois `setLista([])`. Lembre de usar `async` na função.
4. **Pegadinha:** limpe, volte para a calculadora, faça uma conta nova e abra o histórico. As contas antigas voltaram? A Home ainda guarda a lista antiga no estado. Faça a Home recarregar o histórico toda vez que a tela ganha foco, com `navigation.addListener('focus', ...)` no lugar do `useEffect` que carrega só uma vez.

Explique na entrega por que a Home precisa recarregar o histórico, e não só a tela de histórico?

Evolução 2: Excluir uma conta específica

Cada linha do histórico ganha um botão "Excluir" que remove só aquela conta. Conceitos: `filter`, imutabilidade do estado e persistência. Faça depois da Evolução 1, porque ela depende da lista virar estado e da Home recarregar ao voltar.

Resultado esperado

- Tocar em "Excluir" numa linha remove só essa linha.
- Ao fechar e abrir o app, a conta excluída não volta.

Pistas

1. Cada item já tem um `id` único. No `renderItem`, coloque a conta e um botão em uma `View` com `flexDirection: 'row'`.
2. Crie a função `excluir(id)` e monte a nova lista com `lista.filter((item) => item.id !== id)`.
3. Atualize a tela com `setLista(nova)` e salve com `await salvar(nova)` (importe `salvar` de `../storage/historico`).

Explique na entrega: por que usamos `filter`, que cria uma lista nova, em vez de remover o item da lista que já está no estado?

Evolução 3: Reutilizar uma conta do histórico

Ao tocar numa linha do histórico, o app volta para a calculadora com o resultado daquela conta já digitado. Conceitos: passar parâmetros de volta com `route.params` e reagir a eles com `useEffect`. É o desafio da dupla mais adiantada.

Resultado esperado

- Faça `2+3=` (resultado 5), abra o histórico e toque na linha.
- O app volta para a Home e o display mostra `5`.
- Digitando `X2` e `=`, a conta continua a partir do 5.

Pistas

1. A `HistoricoScreen` precisa receber também `navigation`: `function HistoricoScreen({ route, navigation })`. Envolva a linha com `TouchableOpacity`.
2. No `onPress`, volte à Home levando o valor: `navigation.popTo('Home', { valor: item.resultado })`. Com o React Navigation 7, `navigate` empilharia uma segunda Home; `popTo` volta para a que já existe.
3. Na `HomeScreen`, receba `route` junto com `navigation`. Crie um `useEffect` que observa `route.params?.valor` e, quando houver valor, chama `setExpressao(valor)`, `setResultado('')` e `setCalculado(false)`.

Desafio extra: toque na mesma conta duas vezes, mudando o display entre os toques. O valor volta? Se não, o que o `useEffect` está observando e o que precisa mudar a cada toque?

Explique na entrega

Entrega

Ao terminar, cada dupla envia o projeto no GitHub e posta o link no Teams. Você não tem permissão de escrita no meu repositório, então o envio vai para um repositório seu.

1. No GitHub, crie um repositório vazio (sem README).
2. No terminal do projeto: `git remote set-url origin https://github.com/SEU_USUARIO/SEU_REPO.git`
3. `git add .`
4. `git commit -m "Evolução 1: limpar histórico"`
5. `git push -u origin main`.

Checklist da dupla

- O app abre no Expo Go sem erro vermelho.
- Evolução 1 funciona e o histórico continua vazio ao reabrir o app.
- Cada pessoa da dupla explicou ao menos uma linha nova.
- O link do repositório foi postado no Teams.